Proyecto de transformación de datos

Caso Google-Play-Store-Apps

Integrantes:

Diego Bustamante
Nicolás Garrido
Felipe espinoza

Programación para la ciencia de datos

Profesora:

Karla Marilyn Roco Ramirez

Sección:

SCY1101_300D

Índice:

Resumen ejecutivo: 3
Análisis exploratorio inicial: 4
Metodología de transformación: 5
Resultados y validación: 6
Conclusiones y recomendaciones: 9

Resumen ejecutivo:

El propósito de este proyecto fue transformar el dataset "crudo" de Google Play Store en un conjunto de datos estructurado y limpio, apto para modelos de aprendizaje automático.

Los resultados incluyen la eliminación exitosa de caracteres especiales en variables críticas (Price, Installs), la gestión de datos faltantes mediante imputación estadística y la normalización de variables numéricas. Se logró un dataset final con integridad técnica y sin sesgos de magnitud.

El alcance total de la limpieza fueron más de 10,000 registros, identificando y corrigiendo inconsistencias de tipo "string" en columnas que requieren tratamiento matemático.
Mediante el uso de un *ColumnTransformer*, se logró centralizar el preprocesamiento, garantizando que el 100% de las variables de entrada estén escaladas (media 0, varianza 1), eliminando cualquier error de interpretación para futuros algoritmos.

A continuación se ve una captura del dataset original donde se ve que *Installs* y *Price* son de tipo object.

```
df.info()

<class 'pandas.core.frame.DataFrame'>
RangeIndex: 10841 entries, 0 to 10840
Data columns (total 13 columns):
 #   Column          Non-Null Count  Dtype
---  ------          --------------  -----
 0   App             10841 non-null  object
 1   Category        10841 non-null  object
 2   Rating          9367 non-null   float64
 3   Reviews         10841 non-null  object
 4   Size            10841 non-null  object
 5   Installs        10841 non-null  object
 6   Type            10840 non-null  object
 7   Price           10841 non-null  object
 8   Content Rating  10840 non-null  object
 9   Genres          10841 non-null  object
 10  Last Updated    10841 non-null  object
 11  Current Ver     10833 non-null  object
 12  Android Ver     10838 non-null  object
dtypes: float64(1), object(12)
memory usage: 1.1+ MB
```

Como se observa en la salida del *ColumnTransformer*, el dataset procesado (df_procesado) se ha transformado en un **NumPy Array**.

```
df_procesado

array([[-0.17642234, -0.06333855, -0.21364667, ...,  0.        ,
         1.        ,  0.        ],
       [-0.17031848, -0.06333855, -0.62541799, ...,  0.        ,
         1.        ,  0.        ],
       [-0.11426258, -0.06333855,  1.0216673 , ...,  0.        ,
         1.        ,  0.        ],
       ...,
       [-0.17653445, -0.06333855,  0.19812465, ...,  0.        ,
         1.        ,  0.        ],
       [-0.17653445, -0.06333855,  0.60989597, ...,  0.        ,
         1.        ,  0.        ],
       [-0.05197826, -0.06333855,  0.60989597, ...,  0.        ,
         1.        ,  0.        ]])
```

Análisis exploratorio inicial:

El dataset original presentaba una caracterización diversa pero con “ruido” técnico. El dataset contenia información sobre miles de aplicaciones con variables tanto categóricas (Category, Content Rating) como numéricas (Rating, Reviews).

Se observó que la variable Rating tiene una tendencia hacia valores altos (entre 4.0 y 4.7), sugiriendo un mercado competitivo o un sesgo de satisfacción. A su vez se detectó que columnas que deberían ser numéricas se importan como objetos (texto) debido a símbolos como "$" y "+", lo que impedía cualquier análisis estadístico inicial profundo.

Otros hallazgos: La columna Rating presentaba la mayor concentración de valores nulos (cerca del 13% del dataset), lo que requirió una estrategia de imputación para no perder volumen de datos significativo.

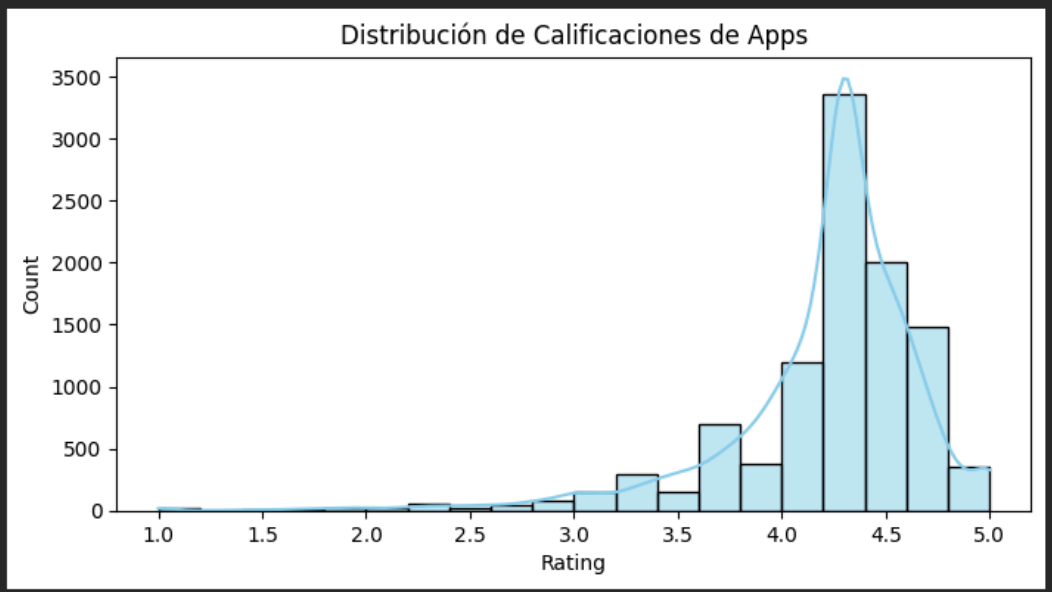
Además de los símbolos "$" y "+", se hallaron registros inconsistentes (como valores no numéricos en columnas de conteo) que habrían generado errores en cualquier pipeline de modelado si no se trataban de forma aislada.

El análisis de las variables cualitativas mostró una alta concentración en categorías como "Family" y "Game", lo que justifica el uso posterior de técnicas de codificación que no asumen un orden de importancia entre ellas.
La imagen a continuación es una prueba sobre lo que llamamos "sesgo de satisfacción".

En el histograma de la variable Rating, se identifica una asimetría. La mayor densidad de datos se concentra en el rango de 4.0 a 4.8, lo que revela un "sesgo de satisfacción" en el dataset.

```python
import matplotlib.pyplot as plt
import seaborn as sns

# Hallazgo 1: Distribución de Ratings
plt.figure(figsize=(8,4))
sns.histplot(df['Rating'], bins=20, kde=True, color='skyblue')
plt.title('Distribución de Calificaciones de Apps')
plt.show()
```



Metodología de transformación:

Para el trabajo se optó por un enfoque de Ingeniería de Características (Feature Engineering), esto quiere decir:

1. Limpieza de Símbolos: Se eliminaron caracteres no numéricos en Installs y Price, con el fin de realizar operaciones matemáticas, para ello estas columnas debían ser de tipo flotante o entero.

2. Imputación de Nulos en Rating: Se utilizó la mediana para rellenar los valores faltantes. *(Justificación: La mediana es más robusta que el promedio ante valores atípicos (outliers), evitando sesgar la calificación general de las apps).*

3. Codificación Categórica (One-Hot Encoding): Se aplicó a variables como Category. Al no existir un orden jerárquico entre categorías (ej. "Juegos" no es mayor que "Finanzas"), esta técnica evita que el modelo interprete una falsa relación de orden.

4. Escalamiento: Se aplicó a las variables numéricas. Esto porque las magnitudes de Installs (millones) y Price (unidades) son muy distintas. El escalamiento nos asegura que el modelo no dé importancia injustificada a la variable con números más grandes.

5. Pipeline de Procesamiento Unificado**:** Implementamos un ColumnTransformer con el objetivo de garantizar la eficiencia y evitar errores manuales. Esta herramienta nos permite aplicar transformaciones divergentes (Escalamiento para números y Codificación para categorías) de forma simultánea sobre el dataframe original.
6. Refinamiento de Limpieza: Limpiamos datos mediante un str,replace, asegurando que columnas como Installs (originalmente con símbolos "+" y ",") se transformaran en vectores de enteros puros.
7. Estandarización: Utilizamos StandardScaler para transformar las variables numéricas a una escala con media 0 y desviación estándar 1.

En la siguiente imagen se ve el bloque de código en el que utilizamos str.replace() para limpiar los simbolos.

Para poder normalizar "Installs" y "Price", primero deben ser números reales, sin texto.

```
# Limpiamos Installs: quitamos '+' y ',' y convertimos a entero
df['Installs'] = df['Installs'].str.replace('+', '').str.replace(',', '').astype(int)

# Limpiamos Price: quitamos '$' y convertimos a float
df['Price'] = df['Price'].str.replace('$', '').astype(float)
```

Resultados y validación:

Tras ejecutar los procesos de ingeniería de características, se logró una conversión exitosa de las variables críticas.

Las variables/columnas numéricas como Installs y Price, que inicialmente presentaban caracteres especiales, fueron transformadas íntegramente a formato numérico. Se validó que no existan valores residuales o errores de conversión (NaN) tras la limpieza.
Aplicamos la mediana para los valores faltantes en Rating permitiendo así mantener la estructura de la distribución original, evitando sesgos artificiales en la media del dataset.

Para confirmar la calidad final del entregable final realizamos dos pruebas fundamentales:

1. Ausencia de Nulos: Se realizó un chequeo post-procesamiento confirmando que el dataset final cuenta con cero valores faltantes, garantizando que cualquier algoritmo posterior pueda procesar la matriz completa
.
2. Estructura del Output (Matriz de Características): Como evidencia definitiva, el resultado del ColumnTransformer se presenta como un NumPy Array de tipo float64. Esta matriz unifica todas las transformaciones (escalamiento y codificación), eliminando cualquier rastro de datos tipo "string" o inconsistencias de magnitud.

Evidencias visuales en el notebook:

Como pruebas fundamentales del trabajo realizado elegimos las siguientes 3 imágenes:
En esta imagen se observa que todos los datos han sido convertidos a una escala uniforme y formato float64, listos para el modelamiento.

```
df_procesado

array([[-0.17642234, -0.06333855, -0.21364667, ...,  0.        ,
         1.        ,  0.        ],
       [-0.17031848, -0.06333855, -0.62541799, ...,  0.        ,
         1.        ,  0.        ],
       [-0.11426258, -0.06333855,  1.0216673 , ...,  0.        ,
         1.        ,  0.        ],
       ...,
       [-0.17653445, -0.06333855,  0.19812465, ...,  0.        ,
         1.        ,  0.        ],
       [-0.17653445, -0.06333855,  0.60989597, ...,  0.        ,
         1.        ,  0.        ],
       [-0.05197826, -0.06333855,  0.60989597, ...,  0.        ,
         1.        ,  0.        ]])
```

En la siguiente imagen se representa la efectividad del preprocesamiento. El dataset original contaba con 13 columnas (que mezclaban texto y números), mientras que el dataset procesado escaló a 38 columnas. Esto es resultado del One Hot Encoding, que transformó las variables categóricas en múltiples vectores binarios.

```
df_procesado.shape

(10346, 38)

df.shape

(10346, 13)
```

Validación de nulos:
En las siguientes imágenes vemos la diferencia entre los valores nulos que existían en el dataset original y el dataset procesado.

Original:

```python
print(df.isnull().sum())
```

```
App                 0
Category            0
Rating           1465
Reviews             0
Size                0
Installs            0
Type                1
Price               0
Content Rating      1
Genres              0
Last Updated        0
Current Ver         8
Android Ver         3
dtype: int64
```

Procesado:

```
Valores nulos por columna:
App               0
Category          0
Rating            0
Reviews           0
Size              0
Installs          0
Type              0
Price             0
Content Rating    0
Genres            0
Last Updated      0
Current Ver       0
Android Ver       0
dtype: int64

Cantidad de duplicados: 0
```

Conclusiones y recomendaciones:

En el desarrollo de este proyecto confirmamos que el preprocesamiento de datos es una de las etapas más importantes en cualquier proyecto relacionado a la Ciencia de Datos.
La transición de un dataset con 13 columnas heterogéneas a una matriz de 38 variables numéricas estandarizadas demuestra que la calidad de un modelo predictivo depende directamente de la integridad de sus datos de entrada. Se logró transformar información "sucia" y dispersa en una herramienta eficaz para el análisis posterior.

Dificultades encontradas:

La limpieza de las columnas Installs y Price representó un desafío técnico, ya que los datos tipo string contenían múltiples caracteres especiales ($, +, ,) que no permitían operaciones aritméticas.

La decisión de usar la mediana para los Ratings fue estratégica para no alterar la distribución natural de los datos, la cual ya presentaba un sesgo de satisfacción hacia valores altos.

Pasos a seguir:

Con el dataset ya procesado y convertido en un array de tipo float64, el siguiente paso podría ser la implementación de un modelo que, por ejemplo, pueda predecir el éxito (Rating) de una app basada en su categoría y precio.